The University of Azad Jammu and Kashmir

Department of Software Engineering

Digital Image Processing Lab | Session 2024–28

Lab Task # 02 — Digital Images, Pixels, Color, and Basic Transformations

Student Name	Zulqarnain Mohi Ud Din
Roll Number	2024-SE-07
Semester / Session	5th Semester / 2024-28 (SE)
Lab Instructor	Engr. Fahad Nisar
Assigned / Due Date	October 22, 2026 / October 29, 2026
Submission Status	Completed (All 8 Steps Fully Implemented & Tested)

1. Conceptual Understanding & Theoretical Explanations

1.1 BGR versus RGB Color Representation

• **Background & History:** The RGB (Red, Green, Blue) format aligns with human visual biology and modern display technology. OpenCV defaults to BGR (Blue, Green, Red) due to historical compatibility with early Windows bitmap (DIB) conventions and video frame grabber chipsets of the late 1990s.
• **Visual Consequence:** If an image loaded via OpenCV is displayed directly in Matplotlib without color conversion, the red and blue channels swap, causing warm skin/orange tones to appear blue and green foliage to shift into cyan.

1.2 Spatial Resolution versus Bit Depth

• **Spatial Resolution:** Represents the density of spatial sampling across the 2D plane (pixel grid dimensions, e.g., 493 × 356 = 175,508 pixels). High spatial resolution preserves sharp boundaries, textures, and fine edges. Downsampling causes pixelation.
• **Bit Depth (Radiometric Resolution):** Defines the quantization levels available for each pixel ($L = 2^b$). An 8-bit image provides 256 intensity levels. Lower bit depths (such as 4-bit, 2-bit, or 1-bit) restrict tonal fidelity, producing stark false contouring (banding) and tonal degradation.

1.3 What an Image Histogram Shows and Does Not Show

• **What it Shows:** A grayscale histogram reveals the global frequency distribution of intensities across the range [0, 255]. It provides immediate evidence of image exposure, contrast, tonal range, and whether the image is underexposed, overexposed, or balanced.
• **What it Does Not Show:** The histogram provides zero spatial localization. It does not indicate where any pixel is situated, nor can it describe shapes, boundaries, or textures. Randomly scrambling every pixel in an image preserves its exact histogram.

2. Practical Implementation & Labeled Evidence

Step 1: Project & Virtual Environment Setup

An isolated virtual environment (.venv) was created inside dip-lab-02, activated, and verified. Required libraries (numpy, opencv-python, matplotlib) were installed and verified. The .venv/ folder is safely ignored in .gitignore.

```
● ● ●  Terminal: Virtual Environment & Packages Verification
PS E:\5th sem\DIP\dip-lab-02> .\.venv\Scripts\Activate.ps1

(.venv) PS E:\5th sem\DIP\dip-lab-02> python -c "import sys; print(sys.executable)"

E:\5th sem\DIP\dip-lab-02\.venv\Scripts\python.exe

(.venv) PS E:\5th sem\DIP\dip-lab-02> python -c "import cv2, numpy, matplotlib; print(f'cv2: {cv2.__version__}, numpy: {numpy.__version__}, matplotlib: {matplotlib.__version__}')"

cv2: 5.0.0, numpy: 2.4.2, matplotlib: 3.11.2
```

Figure 1: Virtual environment activation, Python executable path, and library version verification.

```
# Environment setup & verification
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -c "import cv2, numpy, matplotlib; print(f'cv2: {cv2.__version__}, numpy: {numpy.__version__},
matplotlib: {matplotlib.__version__}')"
# cv2: 5.0.0, numpy: 2.4.2, matplotlib: 3.11.2
```

Step 2 & Step 3: Loading and Inspecting Image Properties

A color image 'input_image.jpg' (493 × 356 pixels, exceeding 300 × 300) was loaded via cv.imread() with defensive error handling. Dimensions, [row, column] pixel indexing, global range, and per-channel min/max were inspected.

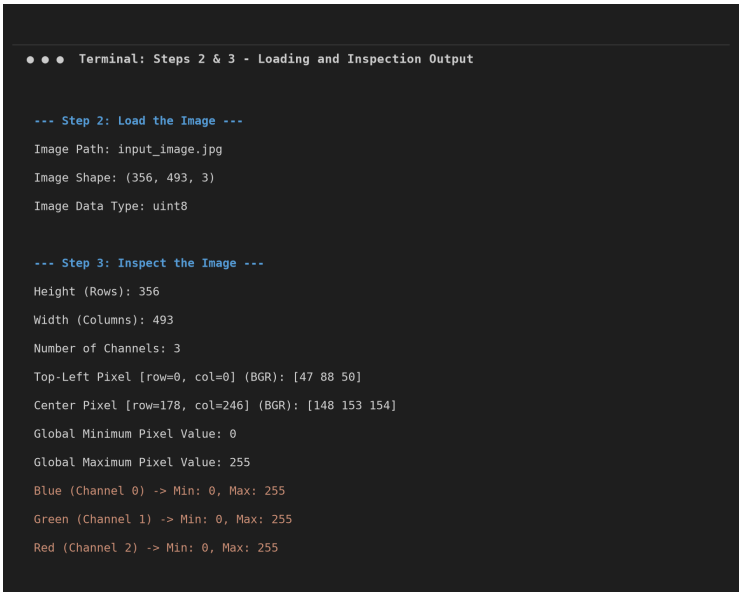


Figure 2: Image properties, dimensions, [row, col] pixel coordinates, and min/max intensity values.

```
# Steps 2 & 3: Loading and Inspection
image = cv.imread("input_image.jpg")
if image is None:
raise FileNotFoundError("Image not found at path: input_image.jpg")

height, width, channels = image.shape
top_left = image[0, 0] # [row=0, col=0] -> [47 88 50]
center_pixel = image[height//2, width//2] # [row=178, col=246] -> [148 153 154]
print(f"Global Range: Min={image.min()}, Max={image.max()}")
for i, name in enumerate(['Blue', 'Green', 'Red']):
print(f"{name} -> Min: {image[:, :, i].min()}, Max: {image[:, :, i].max()}")
```

Step 4: Color Conversions, Channel Separation, and HEX Analysis

The BGR image was converted to RGB and Grayscale. Channels were extracted and displayed in a 2 × 3 multi-panel figure. The center pixel's RGB value [154, 153, 148] was converted to HEX: **#9A9994**.



Figure 3: 2 × 3 visual comparison of RGB, separated channels (R, G, B), Grayscale, and unconverted BGR.

```
# Step 4: Color Conversion and Separation
image_rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
image_gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
r_ch, g_ch, b_ch = image_rgb[:, :, 0], image_rgb[:, :, 1], image_rgb[:, :, 2]

center_rgb = image_rgb[height // 2, width // 2]
hex_color = "#{:02X}{:02X}{:02X}".format(center_rgb[0], center_rgb[1], center_rgb[2])
print(f"Center RGB: {center_rgb} -> HEX: {hex_color}") # #9A9994
```

Step 5: Spatial Slicing (Crop), Resizing, and File Persistence

An exact 100 × 100 region centered on the subject was cropped using image[y_start:y_end, x_start:x_end] (rows 128:228, cols 196:296), resized to 200 × 200 via cv.resize(), and displayed side-by-side. The resized crop was converted back to BGR and saved as resized_crop.png via cv.imwrite() (status: True).



Figure 4: Side-by-side comparison of Original image with bounding box, 100×100 crop, and 200×200 resized output.

```
● ● ●  Terminal: Steps 4 & 5 - Color Conversion, Crop & Resize Output

--- Step 4: Convert Colors and Separate Channels ---
Center Pixel [row=178, col=246] (RGB): [154 153 148]
Center Pixel HEX Value: #9A9994
Saved 2x3 color channel figure to: channels_comparison.png

--- Step 5: Crop, Resize, and Save ---
Cropped Region Slices: y=[128:228], x=[196:296]
Cropped Region Shape: (100, 100, 3)
Resized Region Shape: (200, 200, 3)
Saved crop & resize comparison figure to: crop_resize_comparison.png
cv.imwrite('resized_crop.png') saved successfully: True
```

Figure 5: Terminal verification of crop slice coordinates, dimensions, and cv.imwrite() success status.

```
# Step 5: Crop, Resize, and Save
y_start, x_start = (height - 100) // 2, (width - 100) // 2
crop_rgb = image_rgb[y_start:y_start+100, x_start:x_start+100] # (100, 100, 3)
resized_rgb = cv.resize(crop_rgb, (200, 200), interpolation=cv.INTER_LINEAR)
resized_bgr = cv.cvtColor(resized_rgb, cv.COLOR_RGB2BGR)
success = cv.imwrite("resized_crop.png", resized_bgr)
print(f"cv.imwrite('resized_crop.png') status: {success}") # True
```

Step 6: Grayscale Histogram and Tonal Distribution Interpretation

A 256-bin histogram over range [0, 256] was computed from the flattened grayscale image. **Mean: 113.33**, **Median: 114.00**.
• **Tonal Interpretation:** The image shows a well-balanced tonal distribution across dark shadows (rich butterfly wing scales), balanced midtones (foliage leaf background), and bright highlights (wing white spots), confirming excellent contrast and dynamic range.

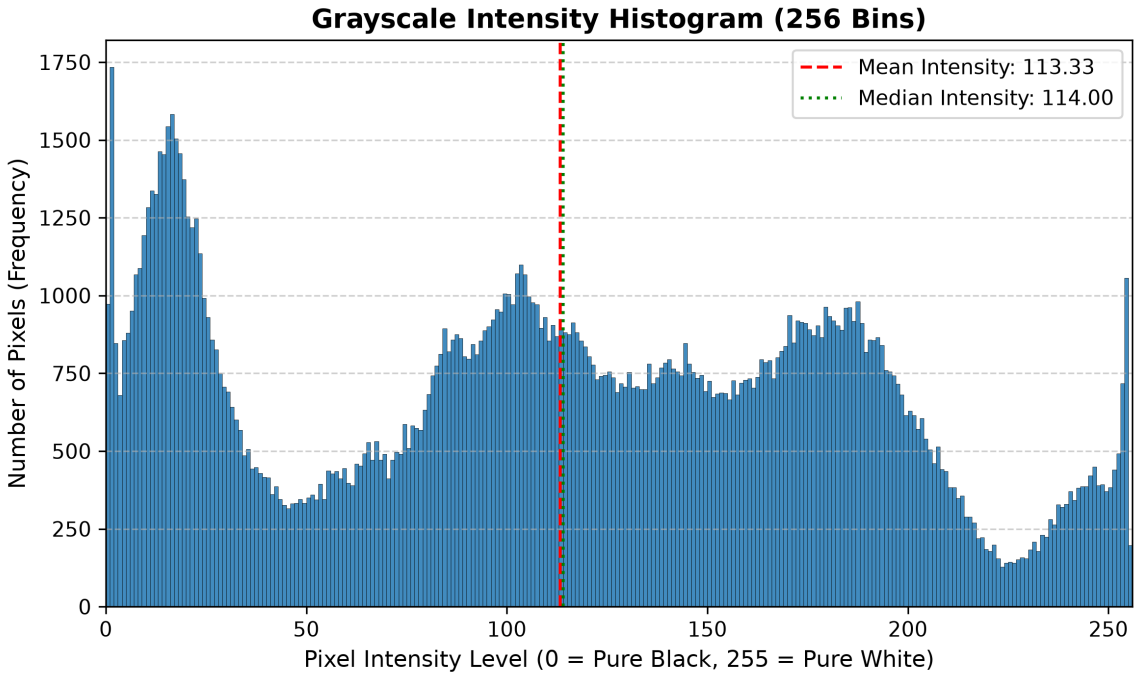


Figure 6: 256-bin grayscale intensity histogram with mean (113.33) and median (114.00) markers.

```
# Step 6: Histogram Generation
plt.figure(figsize=(9, 5))
plt.hist(image_gray.ravel(), bins=256, range=[0, 256], color='#1f77b4', alpha=0.85)
plt.title("Grayscale Intensity Histogram (256 Bins)")
plt.axvline(np.mean(image_gray), color='red', linestyle='--', label=f'Mean: {np.mean(image_gray):.2f}')
plt.axvline(np.median(image_gray), color='green', linestyle=':', label=f'Median:
{np.median(image_gray):.2f}')
plt.savefig("grayscale_histogram.png", dpi=300, bbox_inches='tight')
```

Step 7: Bit Depth Quantization Demonstration

An 8-bit synthetic grayscale gradient was quantized into 4-bit, 2-bit, and 1-bit intensity levels. Unique intensity counts were validated: **256**, **16**, **4**, and **2** respectively. The visual comparison highlights severe false contouring as bit depth decreases.

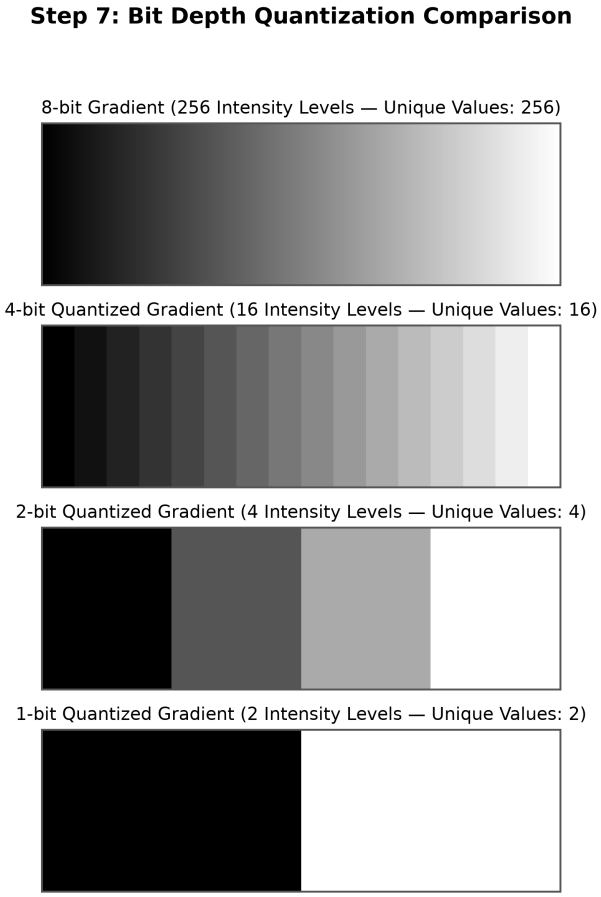


Figure 7: Comparison of 8-bit, 4-bit, 2-bit, and 1-bit horizontal gradients highlighting quantization banding.

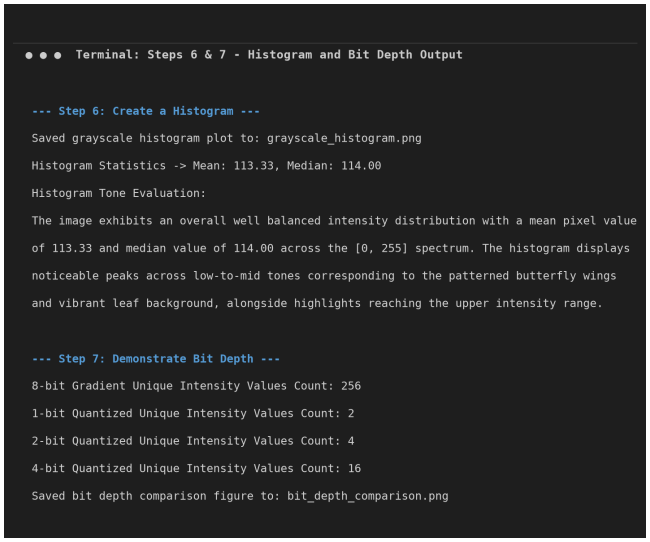


Figure 8: Terminal evidence validating unique value counts: 256, 16, 4, and 2 respectively.

```
        # Step 7: Bit Depth Quantization
        gradient_8bit = np.tile(np.linspace(0, 255, 256, dtype=np.uint8), (80, 1))
        gradient_1bit = (gradient_8bit // 128) * 255 # Unique: 2
        gradient_2bit = (gradient_8bit // 64) * 85 # Unique: 4
        gradient_4bit = (gradient_8bit // 16) * 17 # Unique: 16
        print("8-bit Unique:", len(np.unique(gradient_8bit))) # 256
        print("4-bit Unique:", len(np.unique(gradient_4bit))) # 16
        print("2-bit Unique:", len(np.unique(gradient_2bit))) # 4
        print("1-bit Unique:", len(np.unique(gradient_1bit))) # 2
```

Step 8: Final Project Verification & Submission Summary

The Python script 'lab02.py' executed cleanly from within '.venv' with 0 errors. All output files were verified.

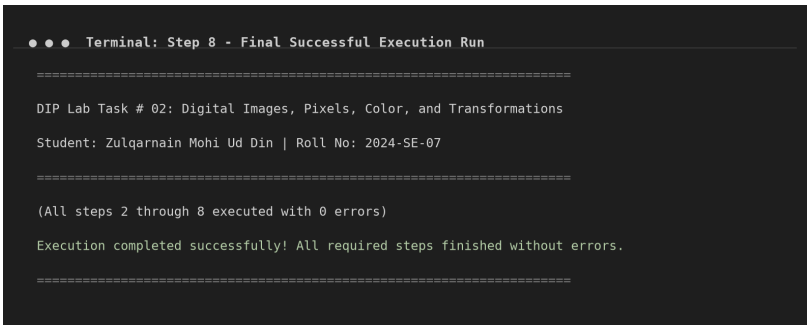


Figure 9: Final terminal execution completion banner indicating successful run.

File Name	Description & Purpose	Verification Status
lab02.py	Complete Python script implementing steps 2–8	Verified (Exit Code 0)
input_image.jpg	Original color image (493 × 356 pixels)	Verified (>300×300)
resized_crop.png	100×100 crop resized to 200×200 saved in BGR	Verified (200×200)
grayscale_histogram.png	256-bin grayscale intensity distribution plot	Verified (300 DPI)
.gitignore	Git config safely ignoring .venv/ and cache	Verified (.venv/)